



TECHNICAL AUDIT & ARCHITECTURE DOSSIER

Comprehensive Codebase Deep-Dive, Technology Inventory, Structural Flaws & Production Roadmap

PROBLEM STATEMENT	TARGET REGION	SDG ALIGNMENT	AUDIT DATE & STATUS
PS-20 (ACM SDG Hackathon)	Pune Metropolitan Testbed	UN SDG 5 & SDG 11	September 2026 · Evaluation Report

1 Executive Summary & System Purpose

SurakshitPath (सुरक्षितपथ) is an infrastructure-driven safe-route planning platform designed to make nocturnal mobility safe, transparent, and predictable for women, students, and late-night workers. Conventional routing engines (Google Maps, Apple Maps) optimize almost exclusively for minimal physical distance or shortest travel time. At night, this algorithmic bias leads commuters through pitch-black alleys, deserted underpasses, and unmonitored service cuts just to save 2 to 3 minutes.

SurakshitPath addresses this gap by replacing pure distance optimization with a **multi-attribute safety impedance model**. It balances street lighting coverage, 24/7 commercial frontage ("natural surveillance"), transit hub access, and proximity to verified emergency safe havens. It aligns directly with **UN SDG 5 (Gender Equality - Target 5.2)** and **UN SDG 11 (Sustainable Cities & Communities - Target 11.2 & 11.7)**, tested across the **Tathawade / Hinjawadi** $\Rightarrow$ **Kothrud** urban corridor in Pune.

2 Complete Technology Stack Inventory

LAYER / DOMAIN	TECHNOLOGIES & LIBRARIES	ROLE IN SURAKSHITPATH ARCHITECTURE
Frontend Framework	React 19.0.0 , React DOM 19.0.0	Core component tree, reactive state hooks, modal layers, dynamic views.
Build Tooling & Type System	Vite 6.2.0 , TypeScript ~5.7.3	Fast ESM bundler, Hot Module Replacement (HMR), static type safety.
GIS & Cartography	Leaflet 1.9.4 , @types/leaflet	Interactive slippy map, vector polylines, pulsing markers, custom glow styling.
Geospatial Engine	@turf/turf 7.2.0	Geodesic calculations, perpendicular cross-track point-to-line distance (\$d_perp\$).
Iconography & UI Tokens	lucide-react 1.16.0 , Vanilla CSS	Nocturnal dark mode design system, semantic tokens, glassmorphic HUD.
Audio & Voice APIs	Browser Web Audio API & SpeechSynthesis	Synthesized 580Hz emergency alert tone, spoken proactive turn-by-turn safety prompts.
Offline Storage	Browser IndexedDB API	Local caching of road network graph nodes, edges, and emergency safe havens.
Backend Microservice	FastAPI 0.110+ , Pydantic 2.0+ , Uvicorn	Python backend service defining scoring, telemetry watchdog, and route endpoints.
Remote Public APIs	Open-Meteo , OSM Nominatim , Overpass QL	Live night weather / visibility (Pune), address search, street lighting queries.

3 What Has Been Built: Core Features & Architecture

 Dual-Route Multi-Attribute Dijkstra

Computes dual routes comparing **Direct Cut (Fastest)** vs **High-Visibility Safe Corridor**. Integrates user-controlled safety tolerance (β) to inflate unlit edge impedance.

 Multi-Factor Infrastructure Scorer

Calculates a transparent safety score (0-100) combining street lighting (L_e), 24/7 commercial shops (A_e), transit hubs (T_e), emergency shelters (E_e), and incident penalties (R_e).

 Explainable AI (XAI) & TreeSHAP

Visualizes safety dimensions via a 4-Factor SVG Concentric Radar Chart and a TreeSHAP waterfall card displaying local feature contributions (ϕ_i) relative to baseline $E[f(x)] = 54.0$.

 Telemetry Watchdog & Anomaly SOS

Applies Extended Kalman Filter (EKF) smoothing on GPS coordinates, monitoring cross-track corridor deviation ($>50\text{m}$) and stationary stalls ($>120\text{s}$) with 45s countdown SOS.

- 1. Edge Impedance Formula: $C(e) = \text{Length}(e) \times [1 + \beta \times (1 - S_e / 100)]$
- 2. Composite Safety Score: $S_e = \text{clamp}[0.35 L_e + 0.25 A_e + 0.20 T_e + 0.15 E_e - 0.25 R_e] \times 100$
- 3. Commercial Frontage: $A_e = \min[1.0, \ln(1 + P0Is) / \ln(1 + 8)]$ | Transit Decay: $T_e = \exp(-\text{dist} / 300\text{m})$

4 Multi-Role Civic & Municipal Ecosystem

USER ROLE	INTERFACE COCKPIT & CAPABILITIES	TARGET BENEFICIARY & CIVIC OUTCOME
Commuter	Google Maps-style 3-column directions, travel mode tabs, priority slider, 1-click safe haven lock, turn-by-turn segment inspector.	Night pedestrians, college students, late-shift IT staff choosing well-lit routes over dark alleys.
Guardian	Live Trip Companion card displaying real-time commuter speed, phone battery level (78%), route envelope safety status, and direct call trigger.	Parents, spouses, and trusted contacts monitoring evening commutes without invasive tracking.
Volunteer	Suraksha Sahayak Network console listing incoming local proximity emergency alerts, estimated arrival distance, and one-tap acknowledge.	Community volunteers, campus security, local shopkeepers acting as rapid first responders.
Civic Admin	Pune Municipal Corporation (PMC & PCMC) SCADA console tracking 1,692 smart poles, dark-spot prioritization, and JSON audit export.	Municipal electrical engineers dispatching targeted streetlight repair work orders.

5 Critical Code Flaws, Deficiencies & Vulnerabilities

CRITICAL BUG 1. TypeScript Production Build Failure (Build Broken)

Running `npm run build` fails with exit code 1. `tsconfig.json` enforces `noUnusedLocals: true`, but `src/components/Map/StreetViewModal.tsx` imports four unused Lucide icons (`Lightbulb`, `Shield`, `Users`, `Video`). The project cannot be deployed or built for production without fixing this.

ARCHITECTURE FLAW 2. Backend Disconnection (FastAPI Service is Dead Code)

While `api/main.py` implements the four documented architecture endpoints, **the React frontend never makes any HTTP requests to it**. Grepping for `VITE_API_BASE_URL` or port `8000` in `src/` yields 0 network calls. Furthermore, `api/main.py` returns static dummy JSON for routes and uses an inaccurate spherical distance calculation that omits `$\cos(\text{lat})$` , causing a ~6% distance distortion.

ALGORITHMIC ILLUSION 3. Pseudo-Machine Learning & Simulated TreeSHAP

The application advertises a "TreeSHAP Local Attribution Model with 94% Confidence". In reality, `mlSafetyModel.ts` contains **no machine learning model, decision tree, or ONNX engine**. It is a set of static TypeScript `if/else` statements adding hardcoded constants (e.g. `+19.5`, `-18.0`) with a hardcoded `0.94` confidence value.

GEOGRAPHIC LIMITATION 4. Hardcoded 16-Node Graph (Cannot Route Outside Corridor)

The road network graph in `puneCorridorGraph.ts` contains only **16 nodes and 20 edges** between Tathawade and Kothrud. If a user selects any location in East or South Pune (e.g., Viman Nagar, Swargate, Hadapsar), `findNearestGraphNode()` snaps to the nearest node in West Pune, producing a route polyline ending kilometers away from the user's destination.

UI BUG 5. Broken Preset Location Button IDs

In `LocationSearchBox.tsx`, preset chips reference `loc_hinjawadi_ph1` and `loc_shivajinagar_metro`. In `geocodingService.ts`, their actual IDs are `loc_hinjawadi_phase1` and `loc_shivajinagar_station`. Because lookup fails, both preset buttons are **silently hidden and never rendered** in the UI.

COMPLIANCE RISK 6. Google Maps Tile Scraping (Terms of Service Violation)

In `MapCockpit.tsx`, Leaflet fetches raw map tiles directly from `https://mt{s}.google.com/vt/lyrs=y`. Scraping Google Maps tile servers without utilizing the Google Maps JavaScript API or Map Tiles API is an explicit violation of Google's Terms of Service and risks IP banning.

HARDCODED MOCK 7. Static Community Hazard Edge Binding & Simulated SOS

In `App.tsx`, when any user submits an infrastructure hazard report anywhere on the map, it is hardcoded to penalize `edge_nh48_tathawade_to_wakad`. Similarly, emergency SOS dispatch in `telemetryWatchdog.ts` executes a `console.log()` and artificial 400ms delay instead of firing real webhooks.

6

Claimed vs. Actual Implementation Reality Matrix

System Capability	Presented / Claimed Mechanism	Actual Codebase Implementation
Routing Engine	City-wide multi-attribute Dijkstra routing across Pune road network.	Fixed 16-node graph hardcoded specifically for Tathawade ⇌ Kothrud.
AI / Machine Learning	TreeSHAP game-theoretic feature attribution on SafetiPin audit regressor.	Static TypeScript <code>if/else</code> heuristic with hardcoded numerical offsets.
Backend API	Live 4-endpoint FastAPI service handling route calculation & telemetry.	Frontend is 100% client-side; Python service is never contacted.
Emergency SOS	Automated n8n webhook dispatch to WhatsApp, Police 112, and guardians.	Client-side <code>console.log()</code> with simulated <code>setTimeout(400)</code> .
Satellite Cartography	Google Maps Satellite Hybrid mapping integration.	Direct raw tile scraping of Google's unofficial tile server endpoints.
Hazard Reporting	Dynamic spatial incident penalty mapped to commuter's report coordinates.	Hardcoded penalty injection onto a single highway link (NH-48).
Multi-Role Network	Distributed network connecting commuters, guardians, and volunteers.	Single-browser local React state toggled by dropdown; no WebSocket sync.

7

Production Engineering & Improvement Roadmap

Phase 1: Immediate Stabilization (Sprint 1)

- **Fix Build:** Remove unused imports in `StreetViewModal.tsx` so `npm run build` succeeds.
- **Fix Presets:** Correct preset landmark IDs in `LocationSearchBox.tsx`.
- **Dynamic Edge Snapping:** Use `Turf.js` `pointToLineDistance` to bind citizen reports to nearest edge.
- **Wire or Retire Dead Code:** Connect `overpassService.ts` or remove unused services.

Phase 2: Scalable Real-World Routing (Sprint 2-3)

- **OSM Road Graph:** Ingest full Pune OSM PBF extract via Valhalla, OSRM, or pgRouting.
- **Custom Edge Costing:** Embed safety formula $C(e) = L \cdot (1 + \beta(1 - S_e))$ into routing engine costing profiles.
- **Compliant Tile Provider:** Migrate to MapLibre GL with vector tiles from OpenFreeMap or Stadia Maps.

Phase 3: True Machine Learning & XAI (Sprint 4-5)

- **Trained Regressor:** Train an XGBoost / LightGBM regressor on SafetiPin urban audit datasets.
- **Browser Inference:** Export model to ONNX format and run in-browser via `onnxruntime-web`.
- **Native TreeSHAP:** Compute mathematical Shapley values using `shap.TreeExplainer`.

Phase 4: Real-Time Network & Mobile Shell (Sprint 6+)

- **WebSocket Sync:** Implement Supabase Realtime or FastAPI WebSockets for guardian trip tracking.
- **Live SOS Dispatch:** Integrate Twilio SMS / WhatsApp Business API for real guardian notification.
- **Native Mobile Wrapper:** Wrap in Capacitor or Flutter for background GPS tracking and screen wake-lock.

